

Laboratory report of Nanjing University of Information Science and Technology Experiment

Title	Lab 4	Date	2025.5.8	Class	24AC
Name	ZhuangZhong	ID	202483910032		

1. Objectives

The main objective of this lab is to understand and practice fundamental Istio service mesh operations on Kubernetes, including:

Installing Istio using istioctl on Windows with Docker Desktop

Deploying Istio's control plane components (istiod, ingress/egress gateways)

Deploying the Bookinfo sample application across multiple microservices

Configuring Istio Gateway and VirtualService for external access

Accessing the Bookinfo web application through the Istio ingress gateway

Understanding key Istio concepts including Gateway, VirtualService, Sidecar proxy injection, and Kiali dashboard

2. Environment Preparation

Istio and kubectl were installed on Windows 11. Docker Desktop with Kubernetes (v1.34.1) was used as the container runtime and Kubernetes platform. The installation was performed using winget (Windows Package Manager) to install Istio version 1.29.2.

After installation, the environment was verified. The following screenshot confirms that the istioctl command is available and that the installation directory has been added to the system PATH.

```
C:\Users\联想>winget install Istio.Istio
已找到 Istio [Istio.Istio] 版本 1.29.2
此应用程序由其所有者授权给你。
Microsoft 对第三方程序包概不负责，也不向第三方程序包授予任何许可证。
正在下载 https://github.com/istio/istio/releases/download/1.29.2/istio-1.29.2-win.zip
29.1 MB / 29.1 MB
已成功验证安装程序哈希
正在提取存档...
已成功提取存档
正在启动程序包安装...
已修改路径环境变量；重启 shell 以使用新值。
添加了命令行别名: "istioctl"
已成功安装

C:\Users\联想>cd istio-1.25.0
The system cannot find the path specified.

C:\Users\联想>where istioctl
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2\istioctl.exe

C:\Users\联想>set PATH=%CD%\bin;%PATH%
```

The Kubernetes context was switched from Minikube to docker-desktop to ensure Istio is installed on the correct cluster. The node status shows a single control-plane node ready.

```
Server: Docker Desktop 4.67.0 (222858)
Engine:
  Version:          29.3.1
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.25.8
  Git commit:       f78c987
  Built:            Wed Mar 25 16:13:48 2026
  OS/Arch:          linux/amd64
  Experimental:     false
containerd:
  Version:          v2.2.1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
runc:
  Version:          1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

```
C:\Users\联想>kubectl config current-context
minikube
```

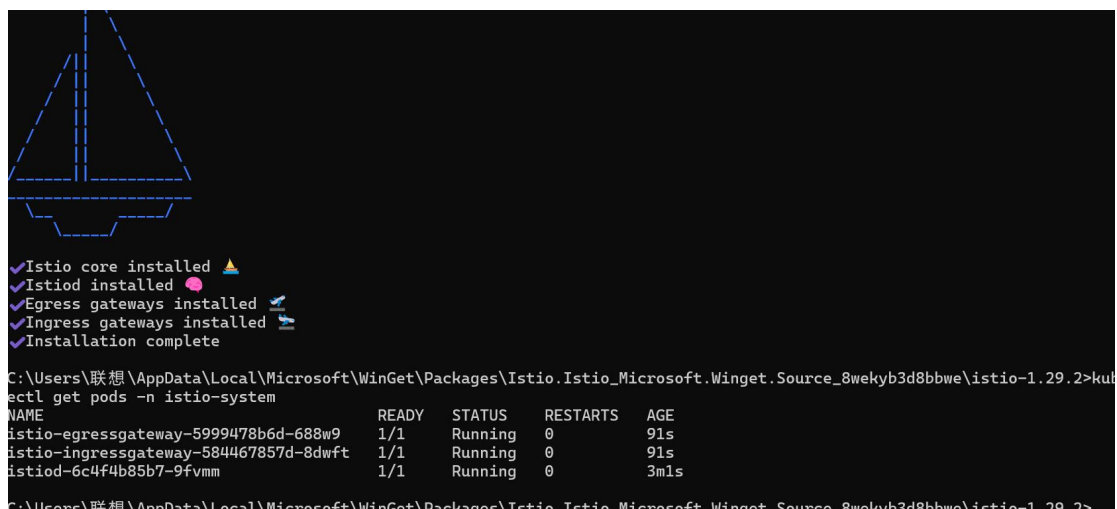
```
C:\Users\联想>kubectl config use-context docker-desktop
Switched to context "docker-desktop".
```

```
C:\Users\联想>kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
docker-desktop   Ready     control-plane   2m33s   v1.34.1
C:\Users\联想>
```

3. Experiment Implementation and Results

3.1 Installing Istio Control Plane

The command `istioctl install --set profile=demo -y` was executed to install Istio with the demo profile. The demo profile includes Istiod, ingress gateway, egress gateway, and telemetry addons. The installation completed successfully.



```

✓ Istio core installed 🚢
✓ Istiod installed 🟡
✓ Egress gateways installed 🛩️
✓ Ingress gateways installed 🛩️
✓ Installation complete

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istio-egressgateway-5999478b6d-688w9 1/1     Running   0           91s
istio-ingressgateway-584467857d-8dwft 1/1     Running   0           91s
istiod-6c4f4b85b7-9fvmm               1/1     Running   0           3m1s
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>
```

3.2 Verifying Istio Pods in istio-system Namespace

After installation, `kubectl get pods -n istio-system` was used to verify all Istio control plane pods are running. The output shows:

```
istio-egressgateway - Running
istio-ingressgateway - Running
istiod - Running
```

All pods show 1/1 READY status, indicating successful deployment.

3.3 Deploying the Bookinfo Sample Application

The Bookinfo sample application consists of four microservices: productpage, details, reviews (three versions), and ratings. All services, service accounts, and deployments were created successfully. The following resources were created: service/ratings, service/reviews, service/productpage

deployment.apps/ratings-v1, reviews-v1/v2/v3, productpage-v1
corresponding service accounts

```
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl get pods -w
NAME                                READY   STATUS              RESTARTS   AGE
details-v1-77d6bd5675-79bd7        1/2     PodInitializing    0           8s
productpage-v1-bb87ff47b-2gm6v     1/2     PodInitializing    0           8s
ratings-v1-8589f64b4c-vfj7b       1/2     PodInitializing    0           8s
reviews-v1-8cf7b9cc5-kmqns        1/2     PodInitializing    0           8s
reviews-v2-67d565655f-d6sbn       1/2     PodInitializing    0           8s
reviews-v3-d587fc9d7-phtjr        1/2     PodInitializing    0           8s
details-v1-77d6bd5675-79bd7        1/2     ErrImagePull       0           13s
details-v1-77d6bd5675-79bd7        1/2     ImagePullBackOff   0           24s
reviews-v3-d587fc9d7-phtjr        2/2     Running            0           60s
reviews-v1-8cf7b9cc5-kmqns        2/2     Running            0           87s
productpage-v1-bb87ff47b-2gm6v     2/2     Running            0           2m3s
reviews-v2-67d565655f-d6sbn       2/2     Running            0           2m24s
ratings-v1-8589f64b4c-vfj7b       2/2     Running            0           3m12s
details-v1-77d6bd5675-79bd7        2/2     Running            0           3m53s
```

3.4 Pod Initialization and Sidecar Injection

After applying the Bookinfo configuration, the pods initially showed 0/2 READY because the Istio sidecar proxy (envoy) is injected alongside the application container. Over approximately 3-4 minutes, all pods transitioned to 2/2 READY and Running status.

The sequence shows:

Initial: 0/2 PodInitializing

Temporary ErrImagePull / ImagePullBackOff (resolved automatically)

Final: all 6 pods 2/2 Running

This demonstrates Istio's automatic sidecar injection working correctly.

```
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl get pods -w
NAME                                READY   STATUS              RESTARTS   AGE
details-v1-77d6bd5675-79bd7        1/2     PodInitializing    0           8s
productpage-v1-bb87ff47b-2gm6v     1/2     PodInitializing    0           8s
ratings-v1-8589f64b4c-vfj7b       1/2     PodInitializing    0           8s
reviews-v1-8cf7b9cc5-kmqns        1/2     PodInitializing    0           8s
reviews-v2-67d565655f-d6sbn       1/2     PodInitializing    0           8s
reviews-v3-d587fc9d7-phtjr        1/2     PodInitializing    0           8s
details-v1-77d6bd5675-79bd7        1/2     ErrImagePull       0           13s
details-v1-77d6bd5675-79bd7        1/2     ImagePullBackOff   0           24s
reviews-v3-d587fc9d7-phtjr        2/2     Running            0           60s
reviews-v1-8cf7b9cc5-kmqns        2/2     Running            0           87s
productpage-v1-bb87ff47b-2gm6v     2/2     Running            0           2m3s
reviews-v2-67d565655f-d6sbn       2/2     Running            0           2m24s
ratings-v1-8589f64b4c-vfj7b       2/2     Running            0           3m12s
details-v1-77d6bd5675-79bd7        2/2     Running            0           3m53s
```

3.5 Configuring Gateway and VirtualService

To enable external HTTP access to the Bookinfo application, an Istio Gateway and VirtualService were created using:

```
bash
```

```
kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
```

The output confirmed:

```
gateway.networking.istio.io/bookinfo-gateway created
```

```
virtualservice.networking.istio.io/bookinfo created
```

Then, port forwarding was set up to access the ingress gateway locally:

```
bash
```

```
kubectl port-forward -n istio-system service/istio-ingressgateway 8080:80
```

This forwards local port 8080 to the ingress gateway's port 80.

```
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>
C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
gateway.networking.istio.io/bookinfo-gateway created
virtualservice.networking.istio.io/bookinfo created

C:\Users\联想\AppData\Local\Microsoft\WinGet\Packages\Istio.Istio_Microsoft.Winget.Source_8wekyb3d8bbwe\istio-1.29.2>kub
ectl port-forward -n istio-system service/istio-ingressgateway 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

3.6 Accessing the Bookinfo Application

The Bookinfo application was accessed by opening

<http://localhost:8080/productpage> in a web browser. The page displays:

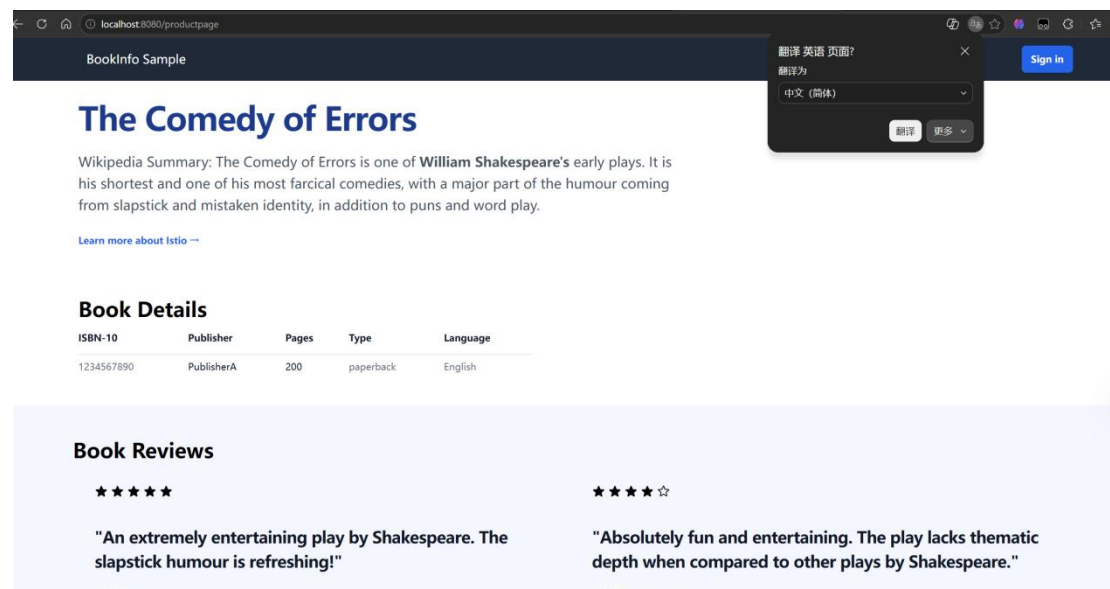
Book title: The Comedy of Errors by William Shakespeare

Book details: ISBN-10, Publisher, Pages, Type (paperback), Language

Book reviews with star ratings (★★★★★ and ★★★★☆)

Sample review texts

The presence of multiple review versions (v1, v2, v3) demonstrates Istio's traffic routing capabilities, though no explicit routing rules were applied yet.



3.7 Kiali Dashboard (Istio Observability)

Kiali is a dashboard for Istio service mesh observability. It was accessed by first installing the Kiali addon (as part of the demo profile) and then accessing it via `istioctl dashboard kiali`. The dashboard shows:

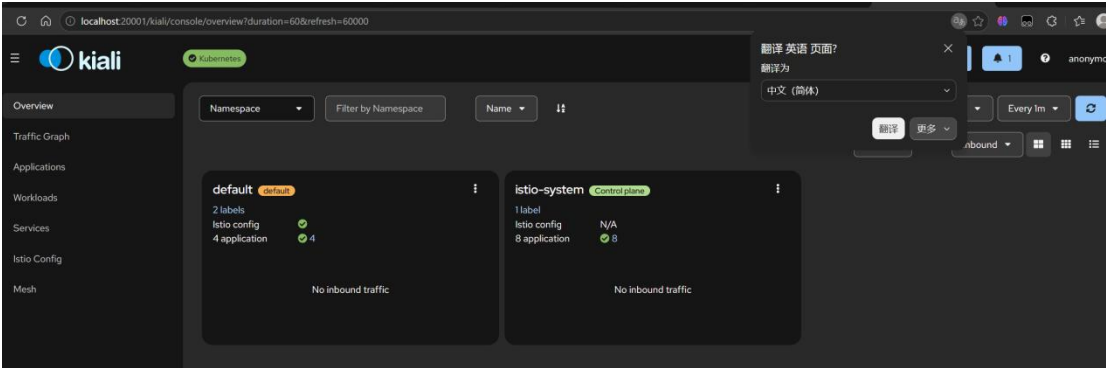
Overview of namespaces: default, istio-system

Traffic Graph showing service relationships

Applications, Workloads, Services views

Istio Config status

Labels and control plane health



4. Issues and Solutions

Several issues were encountered during the experiment, along with their respective solutions.

Issue	Cause	Solution
choco command not found	Chocolatey package manager was not installed on the system	Used winget (Windows Package Manager) instead to install Istio
Path not found cd istio-1.25.0	Wrong version number; actual installed version was 1.29.2	Used where istioctl to locate the correct installation path and set PATH variable
ErrImagePull / ImagePullBackOff for Bookinfo pods	Docker registry rate limiting or network issues on initial pull	Waited ~2-3 minutes; Kubernetes automatically retried and succeeded
No inbound traffic shown in Kiali	Kiali dashboard was opened before generating any mesh traffic	Generated traffic by accessing http://localhost:8080/productpage and refreshing multiple times

Additionally, it is important to note that port forwarding must remain active in a separate terminal window for external access to the ingress gateway. Closing the terminal running kubectkl port-forward will break access to the application.

5. Key Concepts Summary

The core Istio operations covered in this lab include:

Operation	Command	Description
Install Istio	istioctl install --set profile=demo -y	Installs Istio control plane with demo

		profile
Verify installation	kubectl get pods -n istio-system	Checks all Istio components are running
Deploy sample app	kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml	Deploys Bookinfo microservices
Enable sidecar injection	(Automatic via namespace labeling)	Injects Envoy proxy into each pod
Create Gateway	kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml	Exposes services outside the cluster
Port forward	kubectl port-forward -n istio-system service/istio-ingressgateway 8080:80	Forwards local port to ingress gateway
Access application	http://localhost:8080/productpage	Opens Bookinfo web UI
View Kiali dashboard	istioctl dashboard kiali	Launches observability dashboard

Key Istio Concepts Explained:

Service Mesh: A dedicated infrastructure layer for handling service-to-service communication, observability, and security.

Sidecar Proxy (Envoy): A proxy container injected alongside each application pod that intercepts and manages all network traffic.

Istio Gateway: A component that manages inbound and outbound traffic for the mesh, similar to a load balancer.

VirtualService: Defines traffic routing rules (e.g., weights, header-based routing) for services.

Kiali: Observability tool providing topology, health, and metrics visualization for the service mesh.

istio-system Namespace: The dedicated namespace where Istio control plane components run.

6. Conclusion

This lab successfully demonstrated the complete installation and operation of the Istio service mesh on a local Kubernetes cluster (Docker Desktop). The following key achievements were accomplished:

Istio Installation: Istio 1.29.2 was installed using winget and istioctl with the demo profile, providing a complete service mesh environment including ingress/egress gateways.

Bookinfo Deployment: The sample four-tier microservices application was deployed, and Istio's automatic sidecar injection was verified as each pod achieved 2/2 READY status.

External Access Configuration: An Istio Gateway and VirtualService were created to expose the application externally, and port forwarding enabled local browser access.

Application Functionality: The Bookinfo product page was successfully accessed and displayed correct book details, reviews, and ratings, confirming that inter-service communication works correctly through the mesh.

Observability: The Kiali dashboard was launched, providing visibility into the service mesh topology, workloads, and configuration.

Troubleshooting skills were developed by resolving issues including:

Wrong version path (istio-1.25.0 vs istio-1.29.2)

Image pull failures (resolved by automatic retry)

No inbound traffic in Kiali (resolved by generating actual HTTP requests)

The experience gained from this lab establishes a solid foundation for more advanced Istio topics, including:

Traffic Management: Weight-based routing, canary deployments, fault injection, and request timeouts

Security: mTLS (mutual TLS) enforcement, authorization policies, and JWT authentication

Observability: Distributed tracing with Jaeger, metrics with Prometheus/Grafana

Resilience: Circuit breakers, retries, and rate limiting

Multi-cluster mesh: Extending service mesh across multiple Kubernetes clusters